Research Plan Group 1

Statistics 143 - Research in Statistics

Professor Hongquan Xu

Allison Lynn, Ryan So, Cynthia Du, Bryan Mui, Mian Xie

Architecture Design & Hyperparameter Optimization of Convolutional Neural Networks for CIFAR-10 Image Classification

February 19th, 2026

Research Goal

This project aims to develop a convolutional neural network (CNN) that accurately classifies images from the CIFAR-10 dataset into their respective categories. A convolutional neural network is a type of deep learning model specifically designed for processing grid-structured data such as images. Unlike traditional fully connected neural networks, CNNs use convolutional layers to automatically learn spatial hierarchies of features. Examples are edges, textures, and object parts, which are all learned through the application of learnable filters. These filters slide across the input image to detect meaningful patterns while preserving spatial relationships, making CNNs particularly effective for image classification tasks.

This study will systematically explore the impact of hyperparameter tuning on model performance, with the objective of identifying an architecture that maximizes classification accuracy while maintaining computational efficiency. In particular, we will examine architectural and training choices such as the number of convolutional layers, kernel sizes, pooling strategies, activation functions, learning rates, batch sizes, and the number of training epochs.

We plan to compare both computational efficiency and model performance between Neural Architecture Search (NAS) and Differentiable Architecture Search (DARTS) to tune the network architecture, as well as Bayesian Optimization (BO) methods for hyperparameter tuning. By evaluating how these design decisions influence learning behavior and predictive performance, the project seeks to provide a clear justification for each selected parameter in the final model. The results of these experiments will allow us to draw conclusions explaining why certain

hyperparameter configurations lead to improved performance, thereby demonstrating a principled and systematic approach to CNN design for image classification tasks.

Planned Approaches

To begin our research, architecture search and hyperparameter optimization were separated into two distinct stages. Although both influence model performance, they operate at different levels of the learning process. Architecture defines the structural capacity of the model, while hyperparameters govern how that structure is trained. Separating architecture search from hyperparameter tuning allows for a more controlled and systematic optimization process, reducing confounding effects and improving computational efficiency. By first identifying a strong architectural configuration and then refining its training dynamics, we can better attribute performance gains and manage resource constraints.

We first selected Neural Architecture Search (NAS) as the foundational framework for this project in order to systematically automate the design of convolutional neural networks (CNNs). Rather than manually specifying architectural choices, NAS allows us to explore combinations of structural components, including the number of convolutional layers, filter sizes, kernel dimensions, activation functions, and pooling strategies, within a predefined search space. Each sampled architecture is evaluated using validation performance, enabling objective comparison across configurations. While both grid search and random search can be used to explore this space, we initially implemented random search due to limited computational resources. Random

search enables broad exploration while significantly reducing computational cost compared to exhaustive grid search, which becomes infeasible as the search space expands.

To further improve search efficiency and scalability, we also incorporate Differentiable Architecture Search (DARTS), a gradient-based extension of NAS. Unlike traditional discrete NAS methods that require training separate models for each candidate architecture, DARTS relaxes architectural choices into a continuous space by representing operations as weighted mixtures. Using a bilevel optimization framework, network weights are optimized with respect to training loss, while architectural parameters are optimized with respect to validation loss. This simultaneous optimization of weights and architecture can reduce computational requirements while maintaining strong predictive performance. DARTS therefore provides a more efficient and scalable alternative for architecture optimization within our study.

Beyond architectural design, hyperparameter optimization is also critical to maximizing model performance, and our research emphasizes Bayesian Optimization as an efficient and principled approach to hyperparameter tuning. Bayesian Optimization models validation performance as a probabilistic function and constructs a surrogate model often based on a Gaussian process to approximate the relationship between hyperparameters and performance. An acquisition function then selects new configurations by balancing exploration of uncertain regions with exploitation of promising areas. This approach enables the identification of high-performing hyperparameter configurations with fewer evaluations than traditional methods. Together, NAS via random search and DARTS, and Bayesian Optimization provide a structured and computationally efficient framework for optimizing CNN architectures within practical resource constraints.

Overall, the architecture search will focus on convolutional depth, kernel size, pooling type, and activation functions, while Bayesian Optimization will be applied to tune key training hyperparameters such as learning rate and batch size. This combined strategy aims to systematically and efficiently identify high-performing neural network configurations while minimizing computational cost.

Performance Optimization Plan

The performance optimization process begins with an exploration of the CNN architecture prior to hyperparameter tuning. The initial focus is on identifying an effective structural configuration using Neural Architecture Search (NAS). The search space is deliberately constrained to ensure computational feasibility while still allowing meaningful architectural variation. Specifically, the search space includes modifications to the number of convolutional layers, kernel sizes, pooling types, and activation functions. The number of convolutional layers is varied between two and five layers; configurations exceeding five layers are excluded due to the application of pooling at every layer, which would excessively reduce spatial dimensions. Kernel sizes are limited to 3×3 and 5×5 filters, while pooling operations include both max pooling and average pooling. Activation functions considered in the search space are ReLU and Leaky ReLU. Our search space was intentionally designed based on architectural and optimization strategies commonly explored in research literature, reflecting well-established design choices in prior CNN studies.

As a subset of NAS, Differentiable Architecture Search (DARTS) is applied to further refine architectural selection specifically focusing on pooling type, activation functions, and kernel dimensions as the number of layers and filter sizes cannot be altered during DARTS runtime.

Using gradient-based optimization, architectural parameters are updated iteratively alongside network weights. Gradients are evaluated at each step to determine which architectural components are preferred, enabling efficient identification of high-performing configurations without exhaustively retraining separate models.

After establishing a strong architectural foundation, hyperparameter optimization is conducted using Bayesian Optimization. This stage focuses specifically on tuning the learning rate and batch size. By modeling the performance landscape probabilistically and selecting hyperparameter configurations based on expected improvement, Bayesian Optimization enables efficient and systematic refinement of the training process.

Preliminary Results

Manual Exploration of Architecture Types

As an initial step, we manually experimented with different CNN architectures to understand how structural design choices influence classification performance on CIFAR-10. Specifically, we varied the kernel size (3×3 vs. 5×5), the number of convolutional layers (ranging from 2 to 5), and the pooling strategy (max pooling vs. average pooling). This allowed us to isolate the effect of depth, receptive field size, and pooling behavior before implementing automated architecture search methods.

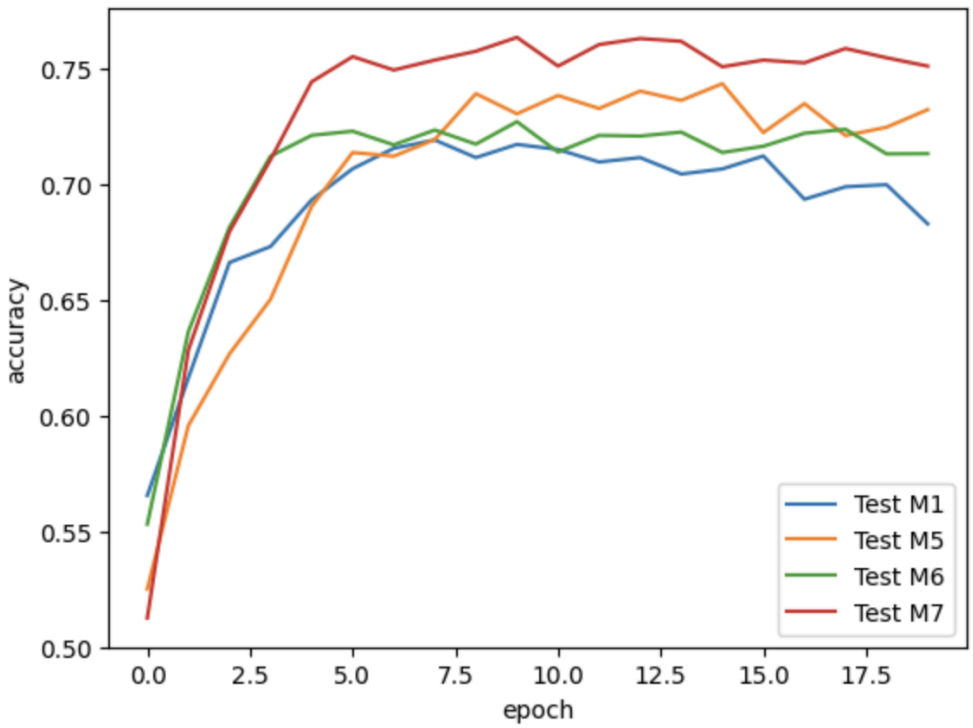

Figure 1

Figure 1 shows the impact of increasing network depth while keeping the kernel size fixed at 3×3 with max pooling (Models M1, M5, M6, M7), where the number of convolutional layers increases incrementally by one in each model, M1 has 2 layers, M5 has 3, M6 has 4, and M7 has 5, to evaluate how additional depth influences performance. As the number of convolutional layers increased from two to five, validation accuracy consistently improved, with M7 achieving the strongest performance and stabilizing around 75–76% accuracy. This suggests that deeper architectures allow the model to learn more hierarchical feature representations, improving generalization performance.

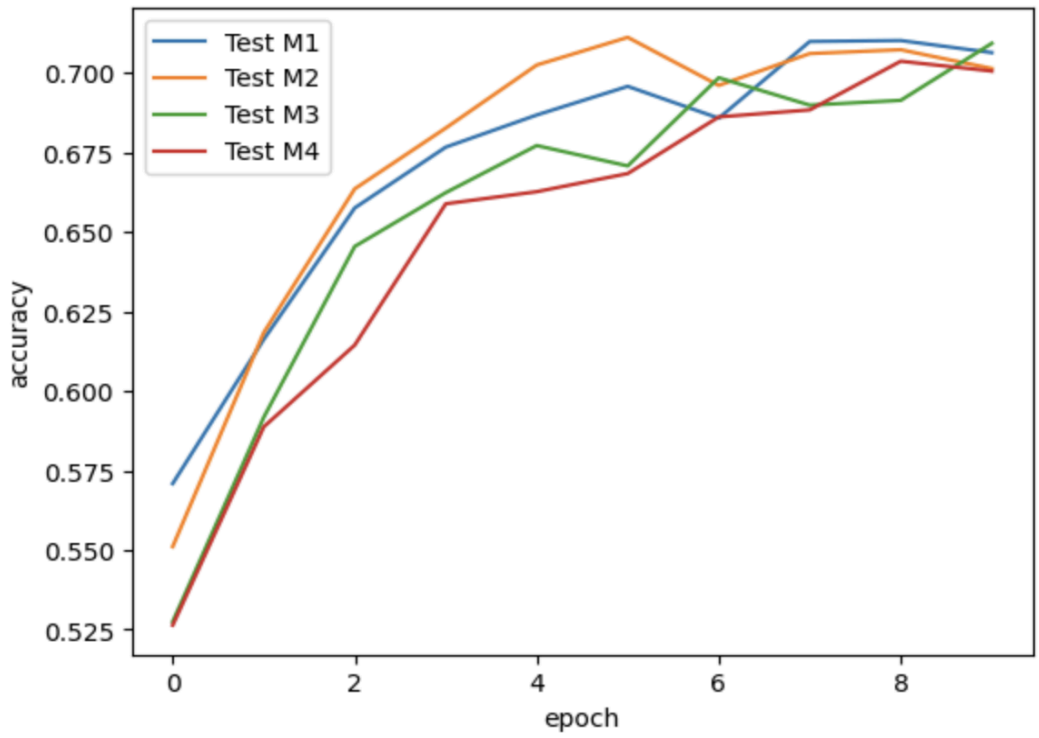

Figure 2

Figure 2 compares kernel size and pooling strategies while keeping the depth fixed at two convolutional layers (Models M1–M4). The results indicate that 3×3 kernels with max pooling (M1) and 5×5 kernels with max pooling (M2) performed similarly, with M2 showing slightly stronger early learning. However, average pooling models (M3 and M4) generally converged more slowly and achieved slightly lower final accuracies. This suggests that max pooling may better preserve discriminative spatial features for CIFAR-10 classification, while larger kernels provide marginal gains at shallow depths but do not dramatically outperform smaller kernels.

Overall, these experiments demonstrate that network depth had a stronger influence on performance than kernel size or pooling choice, motivating the later use of Neural Architecture Search (NAS) and Differentiable Architecture Search (DARTS) to systematically optimize architectural decisions.

Neural Architecture Search (NAS)

As a preliminary step in our Neural Architecture Search (NAS) framework, we defined a search space consisting of the number of convolutional layers, the number of filters per layer, kernel sizes, activation functions, and pooling type. Due to limited computational resources, we implemented Random Search to efficiently sample architectures from this space rather than exhaustively evaluating all possible combinations. Each sampled configuration specified the network depth, per-layer filter dimensions, kernel sizes, activation function (ReLU or LeakyReLU), and pooling strategy (average or max pooling).

Trial	Num Layers	Filters	Kernels	Activation Function	Pooling Type	Training Accuracy	Validation Accuracy
6	3	[64, 64, 64]	[3,3,5]	ReLu	Max	68%	68%
10	2	[16, 64]	[5,5]	ReLu	Max	63%	62%
3	2	[32, 64]	[5,5]	LeakyReLu	Avg	62%	61%
2	4	[16, 32, 16, 64]	[5,5,5,5]	ReLu	Max	60%	60%
5	2	[32, 16]	[3,5]	ReLu	Max	57%	56%
4	4	[64, 32, 16, 16]	[3,5,5,5]	LeakyReLu	Avg	54%	55%
7	2	[32, 16]	[5,3]	ReLu	Max	56%	55%
8	4	[16, 16, 64, 32]	[5,5,3,5]	LeakyReLu	Avg	53%	53%
9	5	[16, 32, 64, 32, 32]	[3,5,3,5,3]	LeakyReLu	Avg	50%	52%
1	2	[16, 16	[3,3]	ReLu	Avg	49%	48%

Table 1

Table 1 presents results from 10 randomly sampled architectures trained for 5 epochs. At this early stage of training, validation accuracy ranged from 48% to 68%, indicating that many models had not yet fully converged. While deeper models and max pooling configurations tended to perform slightly better, performance differences between architectures were less pronounced due to the limited training duration.

Trial	**Num Layers**	**Filters**	**Kernels**	**Activation Function**	**Pooling Type**	**Training Accuracy**	**Validation Accuracy**
10	5	[64, 32, 64, 64, 16]	[5, 5, 5 5, 5]	LeakyReLu	Avg	87%	81%
5	3	[64, 16, 64]	[5, 3, 3]	LeakyReLu	Max	78%	77%
4	3	[32, 64, 16]	[3, 5, 5]	LeakyReLu	Max	81%	77%
6	3	[32, 16, 64]	[3, 5, 5]	LeakyReLu	Max	80%	77%
9	2	[32, 32]	[5, 5]	LeakyReLu	Avg	75%	74%
1	2	[64, 32]	[3, 3]	LeakyReLu	Max	74%	73%
2	4	[32, 32, 32, 32]	[3, 5, 3, 3]	ReLu	Avg	75%	73%
3	5	[16, 32, 32 64 32]	[3, 3, 3, 3, 5]	ReLu	Avg	72%	71%
7	3	[16, 32, 16]	[5, 5,3]	LeakyReLu	Avg	71%	70%
8	3	[32, 32, 16]	[3, 3, 3]	ReLu	Avg	70%	69%

Table 2

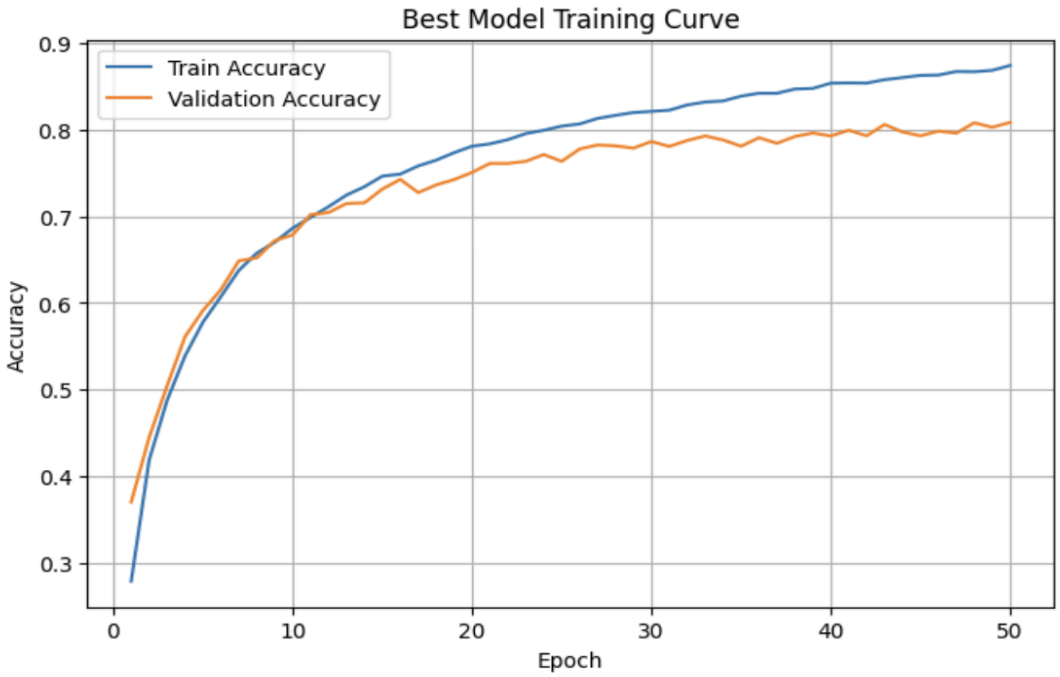

Figure 3

To better evaluate architectural differences, we extended training to 50 epochs (Table 2). With sufficient training time, validation accuracy improved substantially (Figure 3), reaching a maximum of 81%. The strongest-performing architectures generally included larger filter dimensions, suggesting that increased model capacity contributed to improved feature learning. Kernel size, number of layers, and pooling type influenced performance, but their impact was more nuanced compared to filter configuration. These preliminary results suggest that adequate training time and sufficient representational capacity are critical factors in achieving strong classification performance, motivating further exploration of more systematic or gradient-based architecture search methods.

In comparison to our NAS experiments, our initial manual explorations did not yield substantial or reliable conclusions. Because these models were adjusted by hand and trained over a limited number of epochs, performance differences were often inconsistent and difficult to interpret. The small training duration likely prevented full convergence, and the manual selection of architectural parameters limited the scope of exploration. As a result, these preliminary experiments do not provide sufficiently rigorous or scalable insights. For the final analysis, we will prioritize systematic NAS-based methods, which offer a more principled and comprehensive evaluation of architectural configurations.

Potential Problems

During the course of this study, the original research objectives were refined to account for practical computational constraints. Several convolutional neural network architectures reported in the literature require extensive training times, ranging from approximately 1.5 to 4 days of computation time on high-performance GPU hardware, which exceeded the computational resources available for this project. As a result, certain methods, such as NASNet-A and AmoebaNet-A, could not be empirically evaluated due to their runtime requirements. Consequently, the scope of the research was adjusted to emphasize not only classification accuracy but also computational efficiency, allowing for a meaningful and realistic comparison of CNN architectures that can be trained within accessible computing environments. This revised focus ensures that the conclusions drawn remain both practically relevant and scientifically rigorous. We firstly hope to alleviate the computational limitations through utilizing ACCESS resources, a national research cyberinfrastructure ecosystem aimed to provide institutions with

no-cost access to high-performance GPU time. In addition, we hope to modify our analysis methodology by incorporating resource consumption and computation time as metrics when comparing the optimization methods.

References

Access: Advancing science with access. (2024, October 11). Access. https://access-ci.org/

Amit, H. (2024, November 9). *A practical guide to neural architecture search (NAS)*. Medium. https://medium.com/@heyamit10/a-practical-guide-to-neural-architecture-search-nas-166bb6137edc

Bartz, E., Bartz-Beielstein, T., Zaefferer, M., & Mersmann, O. (2023). *Hyperparameter tuning for machine and deep learning with R* (pp. 978–98119–51695). Springer Nature Singapore Pte Ltd.

CIFAR-10: Object recognition in images. (2026). Kaggle. https://www.kaggle.com/competitions/cifar-10/code

Krizhevsky, A. (2009). *CIFAR-10 and CIFAR-100 datasets*. Toronto.edu; University of Toronto. https://www.cs.toronto.edu/~kriz/cifar.html

Kyriakides, G., & Margaritis, K. (2020). *An introduction to neural architecture search for convolutional networks*. https://arxiv.org/pdf/2005.11074

Liu, H. (2025). *Differentiable architecture search for convolutional and recurrent networks*. GitHub. https://github.com/quark0/darts

Liu, H., Simonyan, K., & Yang, Y. (2019, April 23). *DARTS: Differentiable architecture search*. https://arxiv.org/pdf/1806.09055

Pham, H., Guan, M., Zoph, B., Lee, Q., & Dean, J. (2018, February 18). *Efficient neural architecture search via parameter sharing*. https://arxiv.org/pdf/1802.03268

Salehin, I., Islam, Md. S., Saha, P., Noman, S. M., Tuni, A., Hasan, Md. M., & Baten, Md. A. (2023). AutoML: A systematic review on automated machine learning with neural

architecture search. *Journal of Information and Intelligence*, *2*(1). https://doi.org/10.1016/j.jiixd.2023.10.002

Snoek, J., Larochelle, H., & Adams, R. (2012). *Practical Bayesian optimization of machine learning algorithms*. https://arxiv.org/pdf/1206.2944